实验报告：userprog-part2

课程名称：操作系统实践	**年级：**23 级	**上机实践成绩：**
指导教师：张民	**姓名：**叶晓良	
上机实践名称：userprog-part2	**学号：**10235101427	**上机实践日期：**2024/12/30
上机实践编号：	**组号：**	**上机实践时间：**2024/12/30

一、目的

在本实验中，我们的主要目的是深入理解和实现 Pintos 操作系统中的用户程序部分的系统调用功能，特别是围绕文件操作的系统调用，包括文件的创建、打开、关闭和读取。具体目标如下：

1. 理解系统调用机制：
   - 目标：通过实现和分析系统调用，深入理解操作系统如何通过系统调用接口为用户程序提供服务。
   - 方法：研究 Pintos 中系统调用的实现细节，包括如何从用户空间切换到内核空间，以及如何处理和返回结果。
2. 实现文件操作的系统调用：
   - 目标：实现文件创建（create）、文件打开（open）、文件关闭（close）和文件读取（read）的系统调用。
   - 方法：根据 Pintos 的系统调用框架，编写相应的代码，确保这些系统调用能够正确处理用户请求，并在内核中执行相应的文件操作。
3. 确保线程安全：
   - 目标：确保文件操作的系统调用在多线程环境中是线程安全的。
   - 方法：使用文件锁（lock_f）来保护文件操作，防止多个线程同时访问同一个文件时可能出现的竞态条件。

3， 通过测试验证实现：
   - 目标：通过 make check 命令运行测试，验证我们实现的系统调用是否正确。
   - 方法：编写和运行测试用例，确保每个系统调用都能通过相关测试，包括 create、open、close 和 read。

4. 编写详细的实验报告：
   - 目标：记录实验过程中的分析、实现细节、遇到的困难及解决方案，并展示最终的运行结果。
   - 方法：详细描述每个系统调用的分析过程，展示需要修改的代码，解释修改的含义，并提供运行结果的截图或日志。

完成这些目标，掌握 Pintos 操作系统中用户程序部分的系统调用实现，加深对操作系统内核设计和文件系统管理的理解，为今后更复杂的操作系统开发和研究打下坚实的基础。

具体需要实现的目标有：

1. 完成系统调用，使得 make check 通过 create,open,close,read 相关测试

2. 完成实验报告并提交。无需提交代码文件，只需要提交报告的 pdf 即可。在报告中介绍对系统调用的分析过程，展示代码中需要修改的地方，解释修改的含义，和运行结果。

二、内容与设计思想

在本实验中，需要实现一系列的系统调用，包括 halt、create、open、close 和 read，以使 Pintos 操作系统能够通过 make check 测试这些功能。这些系统调用是操作系统与用户程序之间的重要接口，使得用户程序能够请求操作系统执行特定的操作，如文件操作和系统关闭。

1.halt 系统调用

• 函数原型：void halt(void)

• 功能：halt 系统调用用于终止 Pintos 操作系统。这个调用通过 shutdown_power_off() 函数实现，该函数定义在 devices/shutdown.h 头文件中。在实现 sys_halt 函数时，我直接调用了这个函数，因为它已经包含了关机的全部逻辑。

• 参考：

`void halt (void)` [System Call]

Terminates Pintos by calling `shutdown_power_off()` (declared in `'devices/shutdown.h'`). This should be seldom used, because you lose some information about possible deadlock situations, etc.

2. create 系统调用

• 函数原型：bool create(const char *file, unsigned initial_size)

• 功能：创建一个名为 file 的新文件，初始大小为 initial_size。如果创建成功，返回 true，否则返回 false。

• 参考：

`bool create (const char *file, unsigned initial_size)` [System Call]

Creates a new file called *file* initially *initial_size* bytes in size. Returns true if successful, false otherwise. Creating a new file does not open it: opening the new file is a separate operation which would require a `open` system call.

3. open 系统调用

• 函数原型：int open(const char *file)

• 功能：打开一个名为 file 的文件。返回一个非负的文件描述符（fd），如果文件无法打开则返回 -1。文件描述符 0 和 1 分别保留给标准输入和标准输出。

• 参考：

`int open (const char *file)` [System Call]

Opens the file called *file*. Returns a nonnegative integer handle called a "file descriptor" (fd), or -1 if the file could not be opened.

File descriptors numbered 0 and 1 are reserved for the console: fd 0 (`STDIN_FILENO`) is standard input, fd 1 (`STDOUT_FILENO`) is standard output. The `open` system call will never return either of these file descriptors, which are valid as system call arguments only as explicitly described below.

Each process has an independent set of file descriptors. File descriptors are not inherited by child processes.

When a single file is opened more than once, whether by a single process or different processes, each `open` returns a new file descriptor. Different file descriptors for a single file are closed independently in separate calls to `close` and they do not share a file position.

4. close 系统调用

• 函数原型：void close(int fd)

• 功能：关闭文件描述符 fd。进程退出或终止时，会隐式关闭所有打开的文件描述符。

• 参考：

`void close (int fd)` [System Call]

Closes file descriptor *fd*. Exiting or terminating a process implicitly closes all its open file descriptors, as if by calling this function for each one.

5. read 系统调用

• 函数原型：int read(int fd, void *buffer, unsigned size)

• 功能：从文件描述符 fd 所打开的文件中读取 size 个字节到 buffer 中。返回实际读取的字节数（文件末尾时返回 0），如果读取失败则返回 -1。文件描述符 0 从键盘读取。

• 参考：

`int read (int fd, void *buffer, unsigned size)` [System Call]

Reads *size* bytes from the file open as *fd* into *buffer*. Returns the number of bytes actually read (0 at end of file), or -1 if the file could not be read (due to a condition other than end of file). Fd 0 reads from the keyboard using `input_getc()`.

6.文件锁的使用

为了确保文件系统的操作是线程安全的，引入了一个文件锁 lock_f。在进行文件系统操作时，如创建、打开、关闭和读取文件，我会先获取这个锁，操作完成后释放锁。这样可以防止多个线程同时对文件系统进行操作，从而避免数据不一致或损坏。

7.错误处理

在实现这些系统调用时，考虑了错误处理的情况。例如，在 create、open、close 和 read 系统调用中，如果操作失败，返回 -1 来指示错误。此外，在 open 系统调用中，如果文件打开失败，也会返回 -1。

8.测试与验证

为了验证这些系统调用的实现是否正确，运行了 make check 中的相关测试案例，包括创建文件、打开文件、关闭文件和读取文件的测试。通过观察测试结果，确保所有相关测试都通过，从而确认系统调用的正确性。

三、使用环境

本次实验使用 Visual Studio Code (VSCode) 作为开发环境。VSCode 是微软研发的一款免费 IDE，适用于 Windows 和 macOS 系统，尤其适合进行系统开发。相比于 IntelliJ 系列的 IDE，VSCode 不仅免费，而且在远程开发方面有较好的支持。

四、实验过程

1. 系统调用的基本理解

在 Pintos 操作系统中，系统调用是用户程序与操作系统内核之间的接口。通过系统调用，用户程序可以请求内核提供服务，如文件操作、进程管理等。在本实验中，我们需要实现几个关键的文件操作系统调用：create、open、close、read。

2. 文件锁的实现

为了保证文件操作的线程安全，我们引入了文件锁 lock_f。该锁在 thread.c 文件中初始化，并在每个文件操作的系统调用中使用，以确保同一时间只有一个线程能够访问文件系统。

我们翻看相关资料：

> `void halt` (*void*) [System Call]
> Terminates Pintos by calling `shutdown_power_off()` (declared in 'devices/shutdown.h'). This should be seldom used, because you lose some information about possible deadlock situations, etc.

可以知道 halt 函数用于终止 Pintos 操作系统。具体来说：

- 函数声明：void halt(void) 表示这是一个没有参数并且没有返回值的函数。
- 功能：该函数通过调用 power_off() 函数来终止 Pintos 操作系统。power_off() 函数声明在 devices/shutdown.h 头文件中。
- 使用建议：这段文字还提到，halt 函数应很少使用，因为使用它会丢失一些关于可能的死锁情况等信息。这意味着如果系统进入死锁或其他异常状态，使用 halt 函数会直接关闭系统，而不会保留用于调试和诊断这些状态的信息。

总结起来，halt 函数是一个用于终止 Pintos 操作系统的函数，但它应该谨慎使用，以免丢失重要的调试信息。

从而我们编写下面的代码：

1. 头文件包含了 halt 函数需要调用的 shutdown_power_off 函数的声明：

```
6  #include <devices/shutdown.h>
7
```

2. 函数实现：

- sys_halt 是一个系统调用处理函数，当用户程序调用 halt 系统调用时，内核

会调用这个函数。

- struct intr_frame* f 是中断帧的指针，包含了当前中断上下文的信息。
- shutdown_power_off() 是一个关机函数，调用它可以关闭系统。

```
63  // syscall
64  // prototype: void halt(void);
65  void sys_halt(struct intr_frame* f) {
66    shutdown_power_off();
67  }
```

3. acquire_lock_f 和 release_lock_f 是两个函数的声明，分别用于获取和释放一个文件锁：

```
129  void acquire_lock_f(void);
130  void release_lock_f(void);
```

4. 函数实现：

- lock_f 是一个静态的 struct lock 类型的变量，用于表示文件锁。
- acquire_lock_f 函数调用 lock_acquire(&lock_f) 来获取文件锁。
- release_lock_f 函数调用 lock_release(&lock_f) 来释放文件锁。

```
89  static struct lock lock_f;
90  void acquire_lock_f(){
91    lock_acquire(&lock_f);
92  }
93  void release_lock_f(){
94    lock_release(&lock_f);
95  }
```

5. 初始化：

- thread_init 是线程初始化函数，用于初始化操作系统中的线程相关结构。
- ASSERT (intr_get_level () == INTR_OFF) 用于确保在初始化时中断是关闭的。
- lock_init (&tid_lock) 初始化一个用于线程 ID 管理的锁。
- list_init (&ready_list) 和 list_init (&all_list) 分别初始化就绪队列和所有线程列表。
- lock_init(&lock_f) 初始化文件锁。

```
97   void
98   thread_init (void)
99   {
100    ASSERT (intr_get_level () == INTR_OFF);
101
102    lock_init (&tid_lock);
103    list_init (&ready_list);
104    list_init (&all_list);
105    lock_init(&lock_f)
```

Halt 机制：

• halt 函数通过系统调用 sys_halt 调用 shutdown_power_off 来关闭系统。

• 由于关闭系统会丢失调试信息，因此应谨慎使用。

File Lock 机制：

• acquire_lock_f 和 release_lock_f 分别用于获取和释放文件锁。

• thread_init 在初始化时会初始化文件锁，确保在多线程环境下文件操作的互斥性。

这些机制共同确保了 Pintos 操作系统在系统调用和多线程操作中的稳定性和安全性。

3. 创建文件 (create)

create 系统调用用于创建一个新文件。我们通过调用 filesys_create 函数来完成文件的创建，并在操作前后加锁和解锁，以保证线程安全。

我们翻看资料：

> **bool create** (*const char *file, unsigned initial_size*) [System Call]
> Creates a new file called *file* initially *initial_size* bytes in size. Returns true if successful, false otherwise. Creating a new file does not open it: opening the new file is a separate operation which would require a **open** system call.

可以看到 create 函数用于在文件系统中创建一个指定名称和初始大小的新文件。具体来说：

• 函数声明：bool create(const char *file, unsigned initial_size) 表示这是一个返回 bool 类型（true 或 false）的函数，它有两个参数：const char *file：表示要创建的文件名；unsigned initial_size：表示文件的初始大小，以字节为单位。

• 功能：该函数用于在文件系统中创建一个新文件，文件名为 file，并且文件的初始大小为 initial_size 字节。如果文件创建成功，返回 true；如果创建失败，返回 false。

• 使用建议：create 函数通常用于在文件系统中初始化一个新文件，并为其分配初始空间。需要注意的是，创建文件后，如果需要对其进行读写操作，必须单独调用 open 系统调用。此外，如果文件已经存在或文件系统空间不足，create 函数可能会

失败，因此在使用时应检查返回值以处理可能的错误情况。

总结起来就是：create 函数用于在文件系统中创建一个指定名称和初始大小的新文件；创建文件成功返回 true，失败返回 false；创建文件不会自动打开文件，打开文件需要单独调用 open 系统调用。

从而我们编写下面的代码：

1. 实现文件系统的 create 函数，用于在 Pintos 操作系统中创建一个新文件：

1. 代码结构

• 函数名称：sys_create

• 参数：struct intr_frame* f，这是一个中断帧结构体，包含了调用系统调用时的上下文信息，包括堆栈指针（esp）、程序计数器（eip）等。

2. 获取用户堆栈指针：uint32_t *user_ptr = f->esp;

• f->esp 是用户模式下的堆栈指针，指向用户堆栈的顶部。

• user_ptr 是一个指向用户堆栈的指针，用于读取函数参数。

3. 获取文件名和初始大小：f->eax = filesys_create((const char *)*user_ptr, *(user_ptr+1));

• *user_ptr 指向文件名字符串的地址。

• *(user_ptr+1) 指向初始大小的值。

• Filesys_create 是文件系统创建文件的函数，接受文件名和初始大小作为参数，并返回创建是否成功。

4. 加锁和解锁：acquire_lock_f();release_lock_f();

• acquire_lock_f() 和 release_lock_f() 用于在文件系统操作期间获取和释放互斥锁，以确保文件系统操作的原子性，防止多个核心或线程同时访问和修改文件系统数据，导致数据不一致或损坏。

```
69  // syscall
70  // prototype: bool create(const char *file,
    unsigned initial size)
71  void sys_create(struct intr_frame* f) {
72    uint32_t *user_ptr = f->esp;
73    *user_ptr++;
74    acquire_lock_f();
75    f->eax = filesys_create((const char *)
      *user_ptr,
76    *(user_ptr+1));
77    release_lock_f();
78  }
```

4. 打开文件 (open)

open 系统调用用于打开一个文件，并返回一个文件描述符（fd）。每个进程都有独立的文件描述符集合，文件描述符从 2 开始分配。我们使用 filesys_open 函数来打开文件，并将文件描述符与文件对象关联。

我们翻看资料：

> `int open (const char *file)` [System Call]
>
> Opens the file called *file*. Returns a nonnegative integer handle called a "file descriptor" (fd), or -1 if the file could not be opened.
>
> File descriptors numbered 0 and 1 are reserved for the console: fd 0 (`STDIN_FILENO`) is standard input, fd 1 (`STDOUT_FILENO`) is standard output. The `open` system call will never return either of these file descriptors, which are valid as system call arguments only as explicitly described below.
>
> Each process has an independent set of file descriptors. File descriptors are not inherited by child processes.
>
> When a single file is opened more than once, whether by a single process or different processes, each `open` returns a new file descriptor. Different file descriptors for a single file are closed independently in separate calls to `close` and they do not share a file position.

可以看到 open 函数用于打开指定文件，并返回一个文件描述符。具体来说：

• 函数声明：int open(const char *file) 表示这是一个返回 int 类型的函数，它有一个参数：const char *file：表示要打开的文件名。

• 功能：该函数用于打开名为 file 的文件。如果文件成功打开，返回一个非负整数，称为“文件描述符”（fd）；如果文件无法打开，返回 -1。

• 保留文件描述符：文件描述符 0 和 1 是保留给控制台的：fd0（STDIN_FILENO）是标准输入；fd1（STDOUT_FILENO）是标准输出。open 系统调用永远不会返回这两个文件描述符，它们仅在某些特定情况下作为系统调用的参数使用。

• 进程独立性：每个进程都有自己独立的文件描述符集合；文件描述符不会被子进程继承。

• 多次打开文件：当一个文件被多次打开时（无论是同一个进程还是不同进程），每次打开都会返回一个新的文件描述符；对于同一个文件的不同文件描述符，它们的关闭操作是独立的（通过调用 close 函数），并且它们不共享文件位置。

• 使用建议：在打开文件时，应检查返回值是否为 -1，以处理文件无法打开的情况。避免直接使用保留的文件描述符 0 和 1，除非明确需要操作标准输入或标准输出。如果需要多次操作同一个文件，应注意每次打开都会返回独立的文件描述符，并且它们的文件位置是独立的。

总结起来就是：open 函数用于打开指定文件，并返回一个文件描述符。文件描述符 0 和 1 是保留给标准输入和标准输出的。每个进程有独立的文件描述符集合，文件描述符不会被子进程继承。多次打开同一个文件会返回不同的文件描述符，它们的关闭操作和文件位置是独立的。

从而我们编写下面的代码：

1. 数据结构：

首先，代码中定义了一个结构体 thread_file，它包含了文件描述符、文件指针和列表元素：

- fd：文件描述符，一个整数，用于标识打开的文件。
- file：指向实际文件结构的指针。
- file_elem：用于将 thread_file 结构体添加到列表中。

```
132  struct thread_file{
133      int fd;
134      struct file* file;
135      struct list_elem file_elem;
136  }
```

2. 数据结构声明：

此外，在 #ifdef USERPROG 块中，定义了一些与用户进程相关的结构体成员，包括：

- st_exit：退出状态。
- files：文件列表，存储打开的文件。
- file_fd：当前可用的文件描述符编号。

```
 98      #ifdef USERPROG
 99          /* Owned by userprog/process.c. */
100          uint32_t *pagedir;
101          struct semaphore sema;
102          struct thread* parent;
103          struct list childs;
104          struct child * thread_child;
105          int st_exit;
106          struct semaphore sema;
107          bool success;
108          struct thread* parent;
109          struct list files;
110          int file_fd;
111
112      #endif
```

3. 系统调用函数 sys_open：

- 获取用户堆栈指针：uint32_t *usr_ptr = f->esp;f->esp 是用户栈指针，指向用户栈的顶部。
- 获取文件名并打开文件：acquire_lock_f();struct file* file_opened =

filesys_open((const char*)*usr_ptr);release_lock_f();获取文件系统锁，以确保文件操作的原子性。调用 filesys_open 函数打开文件，返回文件指针。释放文件系统锁。

- 处理打开的文件：

```
struct thread *t = thread_current();if (file_opened) {
    struct thread_file *thread_file_temp = malloc(sizeof(struct thread_file));
    thread_file_temp->fd = t->file_fd++;
    thread_file_temp->file = file_opened;
    list_push_back(&t->files, &thread_file_temp->file_elem);
    f->eax = thread_file_temp->fd;} else {
    f->eax = -1;}
```

获取当前线程的结构体。如果文件成功打开：分配 thread_file 结构体；分配文件描述符；将文件指针存储在 thread_file 中；将 thread_file 添加到当前线程的文件列表中；将文件描述符存储在 f->eax 中，作为系统调用的返回值。如果文件打开失败，将 f->eax 设置为 -1。

```
80  // syscall
81  // prototype: int open(const char *file);
82  void sys_open(struct intr_frame* f){
83    uint32_t *usr_ptr = f->esp;
84    *usr_ptr++;
85    acquire_lock_f();
86    struct file* file_opened = filesys_open
      ((const char*)*usr_ptr);
87    release_lock_f();
88    struct thread *t = thread_current();
89    if (file_opened) {
90      struct thread_file *thread_file_temp =
        malloc(sizeof(struct thread_file));
91      thread_file_temp->fd = t->file_fd++;
92      thread_file_temp->file = file_opened;
93      list_push_back(&t->files, &
        thread_file_temp->file_elem);
94      f->eax = thread_file_temp->fd;
95    }
96    else {
97      f->eax = -1;
98    }
99
100 }
```

5. 关闭文件 (close)

close 系统调用用于关闭一个文件描述符。我们通过查找文件描述符对应的文件对象，然后调用 file_close 函数关闭文件，并从文件列表中移除该文件对象。

我们翻看资料：

`void close (int fd)` [System Call]

Closes file descriptor *fd*. Exiting or terminating a process implicitly closes all its open file descriptors, as if by calling this function for each one.

可以知道 `close` 函数用于关闭文件描述符。具体来说：

• 函数声明：`void close(int fd)` 表示这是一个接受一个整数参数（文件描述符）并且没有返回值的函数。

• 功能：该函数通过关闭指定的文件描述符 `fd` 来释放与该文件描述符关联的资源。当一个进程退出或终止时，所有打开的文件描述符都会隐式关闭，就像为每个文件描述符调用 `close` 函数一样。

• 使用建议：这段文字还提到，`close` 函数应在不再需要文件描述符时显式调用，以确保资源被及时释放。此外，进程退出或终止时会自动关闭所有打开的文件描述符，因此在这种情况下不需要显式调用 `close` 函数。

总结起来就是：close 函数的主要作用是关闭指定的文件描述符，释放与其关联的资源。这包括断开文件与进程之间的连接，并允许其他进程使用该文件描述符。

从而我们编写下面的代码：

1. sys_close 函数用于关闭指定的文件描述符：

• 获取用户栈指针：uint32_t *usr_ptr = f->esp;f->esp 是中断帧中的栈指针，指向用户栈的顶部。usr_ptr 指向用户栈中传递的参数（文件描述符）。

• 调整栈指针：usr_ptr++;这行代码跳过系统调用号，使 usr_ptr 指向第一个参数（文件描述符 fd）。

• 查找文件描述符：struct thread_file *opened_file = find_file_id(*usr_ptr);调用 find_file_id 函数，根据文件描述符 fd 查找对应的 thread_file 结构体。如果找到，opened_file 指向对应的 thread_file 结构体；否则为 NULL。

• 关闭文件：

```
if (opened_file) {
   acquire_lock_f();
   file_close(opened_file->file);
   release_lock_f();
   list_remove(&opened_file->file_elem);
   free(opened_file);}
```

如果找到文件描述符对应的 thread_file 结构体：获取文件系统锁（acquire_lock_f()），确保文件操作的原子性。调用 file_close 函数关闭文件。释放文件系统锁（release_lock_f()）。从当前线程的文件列表中移除 thread_file 结构体（list_remove(&opened_file->file_elem)）。释放 thread_file 结构体占用的内存（free(opened_file)）。

```
102  // syscall
103  // prototype: void close(int fd)
104  void sys_close(struct intr_frame* f) {
105    uint32_t *usr_ptr = f->esp;
106    usr_ptr++;
107    struct thread_file *opened_file=find_file_id
       (*usr_ptr);
108    if (opened_file) {
109      acquire_lock_f();
110      file_close(opened_file->file);
111      release_lock_f();
112      list_remove(&opened_file->file_elem);
113      free(opened_file);
114    }
115  }
```

2. find_file_id 函数用于根据文件描述符 fd 查找对应的 thread_file 结构体：

• 初始化变量：struct list_elem *e;struct thread_file *thread_file_tmp = NULL;struct list *files = &thread_current()->files;e：用于遍历文件列表的迭代器。thread_file_tmp：临时存储当前遍历到的 thread_file 结构体。files：指向当前线程的文件列表。

• 遍历文件列表：

```
for (e = list_begin(files); e != list_end(files); e = list_next(e)) {
  thread_file_tmp = list_entry(e, struct thread_file, file_elem);
  if (fd == thread_file_tmp->fd)
    return thread_file_tmp;}
```

从文件列表的头部（list_begin(files)）开始遍历。使用 list_entry 函数获取当前 list_elem 对应的 thread_file 结构体。检查当前 thread_file 结构体的文件描述符 fd 是否与目标文件描述符匹配。如果匹配，返回对应的 thread_file 结构体。

• 未找到文件描述符：return false;如果遍历完文件列表仍未找到匹配的文件描述符，返回 false（通常为 NULL）。

```
117  // find file by file id
118  struct thread file* find_file_id(int fd) {
119    struct list elem *e;
120    struct thread file *thread_file_tmp = NULL;
121    struct list *files = &thread_current ()
       ->files;
122    for(e=list_begin(files);e!-=list_end(files);
       e=list_next(e)){
123      thread_file_tmp=list_entry(e, struct
         thread_file, file elem) ;
124      if(fd == thread_file_tmp->fd)
125        return thread_file_tmp;
126    }
127    return false;
128  }
```

6. 读取文件 (read)

read 系统调用用于从文件中读取数据。对于文件描述符为 0 的情况，我们直接从键盘读取输入；对于其他文件描述符，我们从对应的文件中读取数据。

我们翻看资料：

> `int read` (*int* **`fd`**, *void* ***`buffer`**, *unsigned* **`size`**) [System Call]
> Reads *size* bytes from the file open as *fd* into *buffer*. Returns the number of bytes actually read (0 at end of file), or -1 if the file could not be read (due to a condition other than end of file). Fd 0 reads from the keyboard using `input_getc()`.

可以知道 read 函数用于从文件描述符读取数据。具体来说：

- 函数声明：int read(int fd, void *buffer, unsigned size) 表示这是一个接受三个参数（文件描述符、目标缓冲区、读取字节数）并且返回一个整数的函数。
- 功能：该函数从文件描述符 fd 对应的文件中读取最多 size 字节的数据，并将其存储到 buffer 中。如果文件描述符 fd 为 0（标准输入），则从键盘读取数据，使用 input_getc() 函数实现。
- 使用建议：这段文字还提到，read 函数通常用于从文件或标准输入读取数据。对于文件描述符 0（标准输入），read 函数应特别注意处理键盘输入的情况。这意味着如果系统需要从键盘读取数据，应该使用 input_getc() 函数来实现键盘输入的读取。

总结起来就是：成功时，返回实际读取的字节数（可能在文件末尾读取到 0 字节）。如果读取失败（例如文件无法读取且不是因为文件末尾），返回 -1。

从而我们编写下面的代码：

1. sys_read 函数用于从文件描述符 fd 读取数据到用户提供的缓冲区 buffer 中，读取的字节数为 size：

• 获取用户栈指针：

```
uint32_t *usr_ptr = f->esp;
usr_ptr++;
```

f->esp 是中断帧中的栈指针，指向用户栈的顶部。usr_ptr 指向用户栈中传递的第一个参数（文件描述符 fd）。

• 解析参数：int fd = *usr_ptr;uint8_t *buffer = (uint8_t*)*(usr_ptr+1);off_t size = *(usr_ptr+2);fd：文件描述符，取自用户栈的第一个参数。buffer：目标缓冲区，取自用户栈的第二个参数（指针类型）。size：需要读取的字节数，取自用户栈的第三个参数。

• 处理标准输入（文件描述符为 0）：

```
if (fd == 0) {
   // keyboard input
   for (i = 0; i < size; i++)
      buffer[i] = input_getc();
   f->eax = size;}
```

如果文件描述符 fd 为 0（标准输入），则从键盘读取数据。循环从键盘读取 size 个字节，存储到 buffer 中。返回值设置为 size，表示成功读取的字节数。

• 处理文件读取（其他文件描述符）：

```
else {
   // file read
   struct thread_file *thread_file_tmp = find_file_id(fd);
   if (thread_file_tmp) {
      acquire_lock_f();
      f->eax = file_read(thread_file_tmp->file, buffer, size);
      release_lock_f();
   } else {
      f->eax = -1;
   }}
```

如果文件描述符 fd 不为 0，则调用 find_file_id 查找对应的 thread_file 结构体。如果找到对应的 thread_file 结构体：获取文件系统锁（acquire_lock_f()），确保文件操作的原子性。调用 file_read 函数从文件中读取数据到 buffer 中，读取的字节数为 size。

• 释放文件系统锁（release_lock_f()）：返回值设置为 file_read 的返回值，表示实际读取的字节数。如果未找到对应的 thread_file 结构体，则返回值设置为 -1，表示读取失败。

```
130  // syscall, zzw241229
131  // prototype: int read(int fd, void *buffer,
     unsigned size)
132  int sys_read(struct intr_frame *f) {
133    uint32_t *usr_ptr = f->esp;
134    *user_ptr++;
135    int fd = *usr_ptr;
136    int i;
137    uint8_t *buffer = (uint8_t*)*(usr_ptr+1);
138    off_t size =* (usr_ptr+2);
139    if(fd == 0){
140      // keyboard input
141      for(i=0;i<size;i++)
142      buffer[i]=input_getc();
143      f->eax=size;
144    }
145    else{
146      // file read
147      struct thread file
148      *thread_file_tmp=find_file_id(*usr_ptr);
149        if(thread_file_tmp) {
150        acquire_lock_f();
151        f->eax=file_read(thread_file_tmp,
           buffer, size);
152        release_lock_f();
153        }
154      else
155        f->eax =- 1;
156    }
157  }
158
```

7. 运行结果

在完成上述系统调用的实现后，我们通过 make check 命令运行测试。测试结果显示，create、open、close、read 相关的测试均通过，表明我们的系统调用实现是正确的。【write

那些部分也做了，但是懒得写实验报告了（）】

```
(Process exit codes are excluded for matching purposes. )
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
FAIL tests/userprog/sc-bad-sp
FAIL tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
FAIL tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
pass tests/userprog/write-stdin
```

五、总结

在本次 Pintos 用户程序第二部分的实验中，完成了几个关键的系统调用，包括 create、open、close 和 read。这些系统调用是操作系统文件系统管理的基础操作，涉及到文件的创建、打开、读取和关闭。通过实现这些系统调用，深入理解了操作系统内核如何与文件系统进行交互，以及如何管理进程的文件描述符。

实验过程中的关键点：

- create：我们实现了 create 系统调用，通过调用 filesys_create 函数来创建新文件。在创建文件时，我们使用了一个全局锁 lock_f 来保证文件系统操作的线程安全性。
- open：open 系统调用通过 filesys_open 函数打开一个已经存在的文件，并为该文件分配一个唯一的文件描述符（fd）。每个进程都有独立的文件描述符集合，并且同一文件可以被多次打开，每次都会分配一个新的 fd。
- close：close 系统调用通过调用 file_close 函数来关闭文件，并释放相关的文件描述符。我们还通过 list_remove 函数从当前进程的文件描述符列表中移除已关闭的文件。
- read：read 系统调用根据文件描述符的不同，处理两种情况：读取标准输入（键盘输入）和读取文件内容。对于标准输入，我们使用 input_getc 函数逐字符读取；对于文件内容，我们调用 file_read 函数。
- 在文件操作过程中，我们使用了一个全局锁 lock_f 来防止多个线程同时对文件系统进行操作，确保了文件系统的线程安全性。每次进行文件操作时，我们首先获取锁，操作完成后释放锁。
- 每个进程都有独立的文件描述符集合，我们通过一个链表来管理当前进程的所有打开文件。每次打开文件时，我们为文件分配一个新的文件描述符，并将其添加到链表中。关闭文件时，我们从链表中移除对应的文件描述符。

实验总结与反思

通过本次实验，对 Pintos 的文件系统调用有了深入的理解。掌握了如何实现基本的系统调用，还学会了如何通过锁机制来保证线程安全性。此外，熟悉了如何管理进程的文件描述符，理解了文件描述符与文件之间的映射关系。

在实验过程中，我也遇到了一些挑战。例如，如何有效地管理文件描述符集合，如何处理文件打开和关闭时的线程安全问题。通过仔细阅读代码和反复调试，逐步解决了这些问题，并对操作系统的文件管理有了更深的认识。

总的来说，本次实验巩固了操作系统原理的知识，提高了编程能力和调试技巧。虽然实验结束了，但是期待在未来 CS 生涯中继续深入探索操作系统的其他功能模块。

六、附录

GitHub 地址：Redamancy-Xun/Operating-System-Practice: 操作系统实践 (github.com)